REPORT · 2026年10月26日

H3D 中小型文物三维建模自动化系统 · v1.1.11（r15）

发行说明与路线延伸

H3D 项目组
2026年10月26日

目 录

1 本版说明 4

2 真机测试结果复核 5

3 缺陷一：运行结束，三维预览区是空的 6

3.1 现象 6

3.2 根因链 6

3.3 修法：三层 7

3.4 验证 7

4 缺陷二：交付格式的假告警 8

5 问题一：能不能用管线产出的 PLY 去测路线一 9

5.1 结论 9

5.2 能测到什么 9

5.3 测不到什么，以及会怎么误导 9

5.4 本次真机数据恰好印证了第一、二条 9

5.5 推荐的做法：四步 10

6 X1 与 X2：两条支线的回顾 11

6.1 X1 多时相形变检测 11

6.2 X2 色差评估 11

6.3 两条支线的共同前提 12

7 延伸路线：从「建出来」到「看得懂变化」 13

7.1 X3 病害语义分割与图斑量化 13

7.2 X4 病害语义时序追踪（主线） 13

7.3 X5 修复干预前后评估 14

7.4 X6 轻量化展示交付 15

8 病害语义时序追踪：落地方案 16

8.1 六步任务分解 16

8.2 三个关键设计决策 16
8.3 分期与风险 16
9 验证与发行 **18**
10 下一步 **19**

1 本版说明

v1.1.11（r15）是 1.1.10 真机测试之后的一次小步修正版。**算法一行未改**，改动只有两处缺陷修复，外加一批针对「非 ASCII 路径」的回归用例。本版同时把真机反馈的两个技术问题整理成正式文档：一是能不能用管线自己产出的 PLY 去测路线一，二是 X1 / X2 两条支线的技术基础与可延伸的方向，其中包含病害语义时序追踪（对接 H3D-Lab 分割模块）的落地方案。

项	内容	性质
缺陷一	运行结束后三维预览区空白（项目路径含中文）	功能性缺陷，已修
缺陷二	交付格式报表出现 formats_missing=[glb] 假告警	数据口径缺陷，已修
加固	预览在打开项目时即载入；预览失败不再连累结果条	体验改进
文档	问题一（SfM 模型当扫描件）、问题二（X1/X2 与延伸路线）	本 PDF
测试	全量 856 项通过（1.1.10 为 843，本版净增 13 项）	回归

本版不改算法。1.1.10 的几何与贴图管线在真机上是达标的（覆盖率 81.6%、纹理填充 94.9%、单件 91 秒），本版只处理「看得见的结果」与「报表的措辞」这两类问题。

2 真机测试结果复核

先说结论：管线本身没有退化。下表是 1.1.10 在真机上的实测数据（项目 E:\兵马俑照片\1.1.10，1 件、30 张 4.5 MP 照片、纯照片路线，界面记录总时长 91 秒，各阶段合计 82 秒）。

阶段	耗时	关键指标	解读
S1 稠密重建	0.156 s	n_registered=30，dense=是，s1_reuse.cross_project=是（指纹 7bcb70272bc3）	命中「同照片多件」复用索引，复制了另一项目的 dense 结果，未重新解 SfM
M1 去噪	7.37 s	points_in=622164 → points_out=41486，bg_kept_ratio=0.9186，dense_source=sfm_only	背景剔除保留 91.9% 的点，随后按密度与离群滤到 4.1 万点
M2 配准	0.029 s	n_scans=1，registered=**否**，icp_align.reason=geometry_from_sfm（同坐标系，无需对齐）	纯照片路线下几何只有一源，配准环节空转——这是问题一的关键证据
M3 网格化	2.55 s	59094 顶点 / 117662 面，n_components=14，largest_component_ratio=0.9972	主连通块占 99.7%，碎片化未触发告警
M4 贴图	70.79 s	coverage=59.8%，atlas_util=63.0%，fill=94.9%，projected_cameras=30，texels=2.64 M	图集 2048²，UV 20.6 s + 采样 49.2 s，遮挡剔除 726 万次
M5 质检	0.66 s	coverage=81.6%，中误差=**N/A**，相对中误差 0.73%，清晰度 71.9，接缝 2.34	无尺度锚点，绝对精度按规范置 N/A，只给相对口径
M6 交付	0.22 s	formats_requested=[ply, obj, glb]，实际交付 [ply, obj]	缺 glb —— 见第 4 章，是配置口径问题而非导出失败

三个值得记住的口径：

① **S1 的 0.156 秒不是性能突破**，是复用索引命中（同一批照片在另一个项目里解过）。真正解一次 SfM 的时间要看首次运行。复用是否可信，报表里 s1_reuse.params_match=是 与 fingerprint 都在，可以反查。

② **M2 的 29 毫秒说明配准没有发生。**这不是偷工：纯照片路线下几何只有一个来源，它和照片天然同坐标系，ICP 的对齐量恒等于单位阵。想测配准，必须制造第二个几何源（真实扫描件，或人为错位——见 5.5）。

③ **中误差 N/A 是正确行为。**模型没有尺度锚点时，任何毫米数都是「假设 1 单位 = 1 米」换算出来的，写进论文表格就是伪精度。相对中误差 0.73%（占包围盒对角线）才是这一件唯一站得住的口径。

3 缺陷一：运行结束，三维预览区是空的

3.1 现象

真机截图：进度 100%、状态「√ 生成完成」、阶段耗时条正常，但「3D 模型预览」里只有一个空的坐标框，连「未生成可预览的三维模型」这句提示都没有。同时日志停在最后一行「● 完成：total=1 done=1 failed=0 skipped=0」——之后再没有任何输出。

这三个现象凑在一起就是一条线索：**界面更新在预览这一步停住了。**因为「结论进结果条」的代码排在预览之后，预览抛异常 → 结果条也不出现；而异常没有进日志，说明它不是被我们捕获后记录的，是**逃出 Qt 槽了。**

3.2 根因链

用两行脚本就能复现：在中文目录下直接读交付的 OBJ。

```
import open3d as o3d
o3d.io.read_triangle_mesh(r"E:/兵马俑照片/1.1.10/交付成果/relic_001/relic_001.obj")
# UnicodeDecodeError: utf-8 codec cant decode byte 0xb1 in position 40
# 同一目录的 relic_001.ply 读取正常（0.03 s）
```

0xb1 是什么？是「兵」的 GBK 首字节。Open3D 0.19 对 OBJ / GLB 走 ASSIMP，路径按 ANSI（本机即 GBK）窄字符传给 C++，回程又当 UTF-8 解码，于是**路径里只要有中文就必炸**；而 PLY / 点云走 Open3D 原生 reader，路径由宽字符接口处理，不受影响。

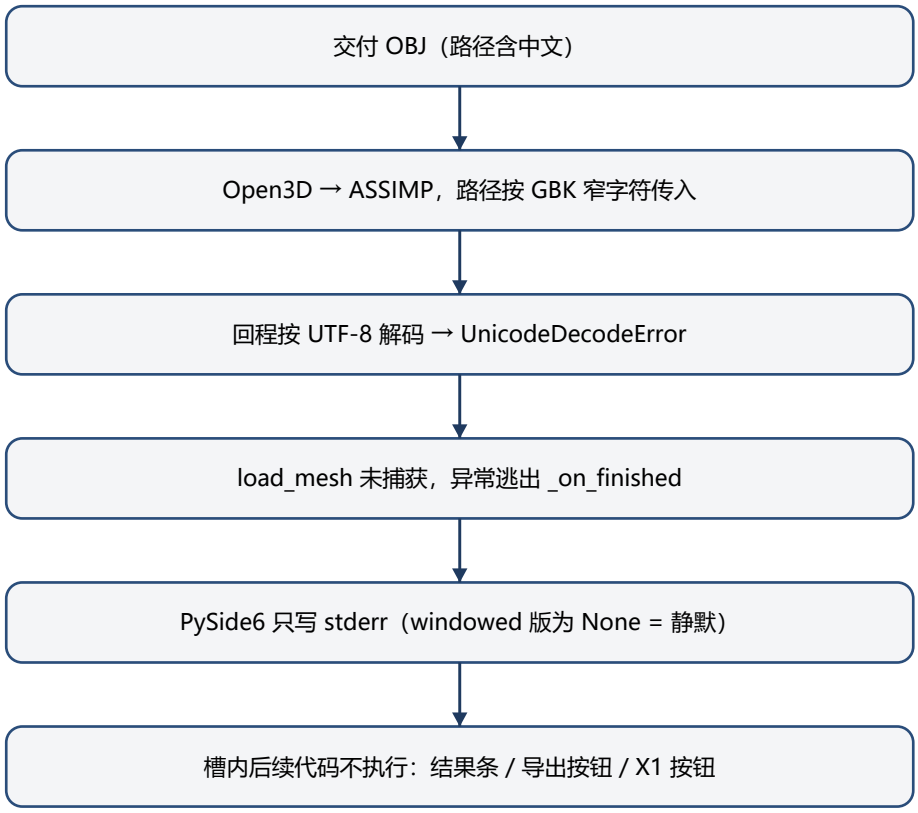


图 3-1 从一次路径解码失败到一个「半成品界面」的链条

> **这一类问题的通病**：Qt 槽里的异常不会抛给调用方，PySide6 只打一行 stderr。源码运行还能在控制台看到，打包成 windowed exe 后 sys.stderr 为 None，连那行都没有。所以「槽内的关键路径必须有 try/except + 可见日志」，否则失败就等于界面少一块，且不留痕迹。

3.3 修法：三层

层	位置	做法	防住什么
① 读	新模块 gui/meshload.py	read_mesh / read_point_cloud / read_image 统一封装：先直读，失败且路径含非 ASCII 时把模型**连同 mtl 与贴图**复制到 ASCII 临时目录重读；按 路径+大小+mtime 缓存	中文、日文、带空格与方括号的项目路径；也顺带覆盖 ASSIMP 读损坏文件
② 用	gui/preview.py、gui/viewer3d.py	所有读取走 meshload，函数内部再兜一层 try/except 返回 False；失败只让预览为空，不往外抛	预览失败污染主流程
③ 兜	gui/app.py	抽出 _refresh_preview()，_on_finished 里单独 try/except 包住并写日志；打开项目时也调一次	任何未来的预览异常都不能吃掉结果条、导出按钮与 X1 按钮

另有两处顺手的改进：预览候选文件顺序改为 **PLY 优先**（原生 reader，几何与 OBJ 完全一致，但不受路径编码影响）；面数超过 6 万时先做体素聚类简化（11.7 万面 → 2.8 万面，重绘 0.50 s → 0.18 s），拖拽旋转不再卡顿，简化只发生在预览里，交付文件一个字节都不动。

```
def read_mesh(path):
    """读三角网格；失败返回 None（绝不抛异常）。"""
    try:
        mesh = o3d.io.read_triangle_mesh(path)
        if mesh is not None and not mesh.is_empty():
            return mesh
    except Exception:          # 非 UTF-8 路径 / ASSIMP 失败
        pass
    if has_non_ascii(path):    # 中文路径兜底：ASCII 副本重读
        alt = ascii_copy(path) # 连 mtl 与贴图一起搬，引用名不合法时改写
        if alt:
            return o3d.io.read_triangle_mesh(alt)
    return None
```

3.4 验证

新增 tests/test_gui_preview_load.py 共 12 项，其中第一项就是**把缺陷本身钉住**：断言「在中文目录下直接读 OBJ 会抛 UnicodeDecodeError」。这样将来 Open3D 修好了这个缺陷，用例会先红一次，提醒我们评估能否删掉副本逻辑。其余覆盖：副本回退能读到 117662 个面、贴图按字节复制、副本缓存、损坏文件返回 None、MeshPreview 不抛异常，以及最关键的一条——**交付物损坏时结果条仍然出现。**

实测：真机项目在源码版与打包版下都立即出图；把 PLY 删掉只留 OBJ，副本回退 1.7 秒读完 11.7 万面。

4 缺陷二：交付格式的假告警

M6 的指标里写着 formats_requested=[ply, obj, glb]、formats_missing=[glb]。也就是说报表认为「用户要 glb，我们没交」。但翻回去看约束确认时用户选的是 **ply + obj**，M4 也只产出了这两种。问题在配置口径：界面把约束里的格式写进了 texture.export_formats（M4 用），却没同步 publish.formats（M6 用），后者一直是模板里的 [ply, obj, glb]。M6 只是从 50_tex 复制，50_tex 里没有 glb，自然就「缺失」。

这不只是文案问题：验收时看到 missing 会以为导出失败，真实原因却是「请求了一个从未打算产出的格式」。修法是让两者同源——交付请求 = M4 产出格式：

```
cfg["texture"]["export_formats"] = self.export_formats
cfg["publish"]["formats"] = list(self.export_formats)
# M6 只从 50_tex 复制；请求 M4 没产的格式，必然报 missing
```

告警本身保留：如果 M4 真的漏写了某个格式，requested 与 delivered 仍会不一致，只是现在 requested 是「用户真选过的」。配套用例锁住了这条，见 test_publish_formats_follow_constraints。

> **取舍说明**：为什么不让 publish.formats 独立于纹理格式？因为 M6 的职责只有「复制 + 归档 + 写元数据」，它没有能力凭空生成格式。让下游声明一个上游不可能提供的清单，就是在制造假指标。

5 问题一：能不能用管线产出的 PLY 去测路线一

5.1 结论

> **可以用，但只能当「功能性回归」，不能当「精度验证」。**SfM 出来的 PLY 能覆盖路线一的 8 个阶段里除配准与尺度之外的 6 个；而路线一存在的意义恰恰是「有真实几何先验 + 有度量基准」，这两点它一条都给不了。正确用法是：拿它做快速、可复现、不依赖硬件的冒烟与回归，再人为构造一次已知变换去测配准，真精度仍必须靠一件真实扫描件。

5.2 能测到什么

环节	能被 SfM 模型覆盖吗	说明
M1 去噪 / 背景剔除	能	点云密度分布与真实扫描不同，但流程、法线朝向投票、聚类剔除都能走通
M2 配准	仅当人为错位	直接喂同一份数据 = 同坐标系 = 恒等变换，测不到 FPFH + RANSAC + ICP；人为错位后可以测（见 5.5）
M2 双源融合判据	能	同一件塞两份几何，可直接验证 novel 判据（novel 小于 0.15 不并入），这是纸面逻辑最容易写错的地方
M3 泊松重建	能	面数、连通块、碎片率都会照实给出
M4 贴图	能	UV 展开、Z-buffer 可见性、多视图加权、泊松溶合全链路
M5 质检 / M6 交付	部分	覆盖率、纹理指标、碎片率都能测；中误差没有意义（见 5.3）

5.3 测不到什么，以及会怎么误导

盲区	后果
配准退化	同源几何与照片天然同坐标系，ICP 误差恒约等于 0，报表会显示「配准完美」。这是最危险的一条：它把没测的东西显示成满分
尺度不可度量	SfM 无标尺。一旦把 PLY 放进 00_raw，程序会认为「有扫描件 = 有度量锚点」，用 scale.reference_diag 去缩放，于是精度表里出现「看起来有量纲、其实是猜的」毫米数
噪声形态不同	激光扫描是均匀、单向、带强度；SfM 稠密点云密度随视角与景深剧烈变化，边缘有飞点。M1 的去噪参数在这两种数据上的表现不可互换
双源问题被掩盖	真实场景要处理「扫描件与照片位姿不一致」，同源数据下这一层完全不存在
硬件依赖被掩盖	扫描件的强度与反射率、多站拼接错层、驱动与 GPU 依赖，SfM 数据一个都碰不到

5.4 本次真机数据恰好印证了第一、二条

M2 耗时 29 毫秒、registered=否、icp_align.reason 直接写着「geometry_from_sfm（同坐标系，无需对齐）」。把它换成 SfM 出的 PLY 塞进 00_raw，这条结论一个字都不会变——因为几何仍然是同一批照

片算出来的。

同时 M5 的 metric_units=否、中误差 N/A。若换成「伪扫描件」路线，程序会因为 00_raw 里出现文件而认为有锚点，这条 N/A 会变成一个具体的毫米数，而那个数字只反映 reference_diag 填得准不准。

5.5 推荐的做法：四步

步	做法	判据
① 冒烟基线	把 SfM 的 fused_dense.ply 拷进 00_raw/<件>/ 跑全链	只看「跑不跑得通、有没有崩、交付物齐不齐」，不看精度数字
② 制造真配准任务	对 PLY 施加已知刚体变换（如绕 Z 轴 15 度、平移 3 cm），再让 M2 恢复	恢复残差小于 2 cm 视为通过；这是唯一能在家测出 FPFH + RANSAC + ICP 的办法
③ 补尺度和锚点	画面里放印刷标尺 / 标定球 / 色卡，或量取三点实际距离写入 reference_diag	有了真实长度，M5 的中误差才有意义
④ 留痕	在 00_raw/<件>/ 放一个 SOURCE.txt 注明「由 SfM 生成、非激光扫描」，报表备注同步带出	半年后仍能分清哪件是扫描件、哪件是合成的

第 ④ 步容易被当成多余的仪式，但它决定论文的可辩护性：评审只要问一句「这一件的精度是怎么来的」，没有留痕就只能靠记忆回答。建议把来源标记做成 manifest 的一等字段，而不是散落的文本文件。

6 X1 与 X2：两条支线的回顾

X1 / X2 都是可选阶段，默认关闭（runner.x1_enabled / x2_enabled），开关打开后排在 M6 之后运行。它们的共同点是**不产生几何，只产生评价**——这也是为什么它们失败只降级、不阻断交付。

6.1 X1 多时相形变检测

X1 回答的问题是「这件东西跟上一期比，变了多少」。输入是上期点云（阶段成果/00_ref/<件>/*.ply）与当期稠密点云（10_dense/<件>/fused_dense.ply），算法是 M3C2——沿局部法向把两期点云投影到同一柱体内比较距离分布，并用点云自身的配准不确定度给出显著性门槛（LoD95）。输出 <件>_X1_m3c2.json（max_change、changed_ratio）并并入 accuracy.csv。

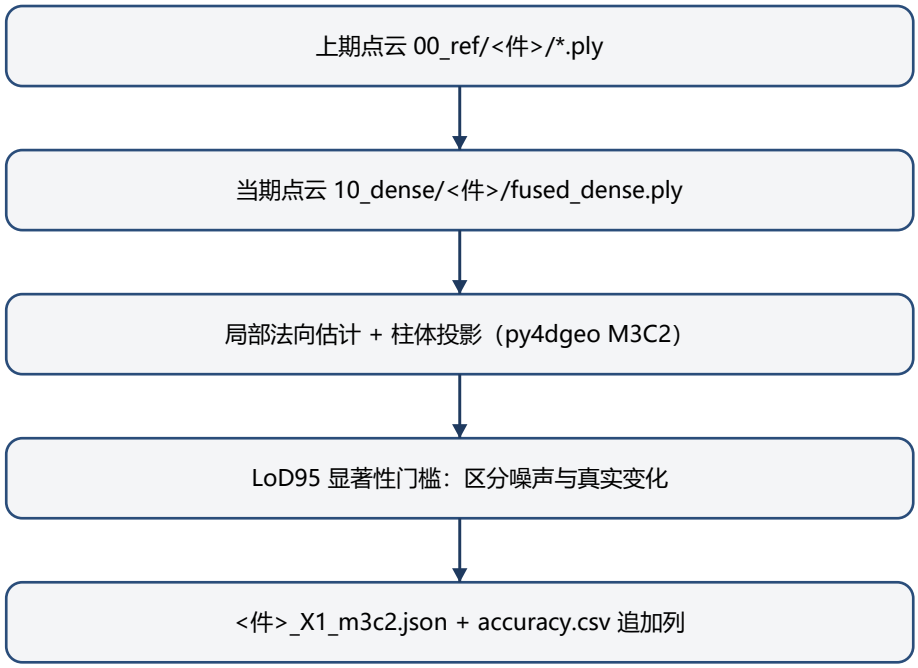


图 6-1 X1 的数据流

技术基础	为什么必需	缺了会怎样
两期同坐标系	M3C2 假设两期已在同一参考系	差分结果全是「位姿差」而不是「形变」
尺度已知	max_change 要报毫米	只能报相对变化，不能跨件比较
共视重叠	柱体投影需要两期在同一位置都有点	重叠区外一律无值，容易被误读成「没变化」
点密度相当	M3C2 的 LoD 与点密度强相关	一期稀一期密时 LoD 被高密度侧拉低，假阳性上升

6.2 X2 色差评估

X2 回答的是「做出来的纹理颜色准不准」。输入是 M4 产出的纹理图集与照片里的色卡；流程是在照片中用 ArUco 四角标记定位 ColorChecker 24 色卡，单应变换取出 24 个色块中心色，与标准参考值算 CIEDE2000。找不到色卡时退回纹理随机采样，此时 checker_source=texture-random，**这批 ΔE 不可引用**（这一点必须在报表里写明，否则会被当成真实色差）。

技术基础	说明
色卡入画	每件至少一张带 ColorChecker 的照片，色卡正对镜头、无遮挡
光照稳定	同一组照片用同一光源与白平衡；混光会让 ΔE 失去意义
色彩空间明确	sRGB / AdobeRGB 必须可追溯，RAW 需固定显影参数
避免高光	金属与釉面高光会污染色块采样，必要时加偏振
CIEDE2000	比 ΔE76 更符合人眼感知，也是文献常用口径

6.3 两条支线的共同前提

X1 和 X2 都在回答「质量」，而质量结论只在**口径明确**时成立：尺度（mm 还是模型单位）、配准（同坐标系还是已对齐）、光照（同组光源还是跨日）、模型（哪一版管线）四个条件缺一不可。报表里之所以把这些字段一个个列出来，就是为了让结论可复核，而不是给一个无法追溯的数字。

7 延伸路线：从「建出来」到「看得懂变化」

现有的 M1–M6 解决「把一件文物变成可用的三维模型」，X1 / X2 解决「这个模型好不好」。再往前一步，用户真正关心的是**病害怎么变**。下面四条路线按落地难度排序，第一条与第二条是主线（与 H3D-Lab 分割模块对接），后两条是支线。

代号	路线	一句话价值	依赖
X3	病害语义分割与图斑量化	把「看起来有裂缝」变成「裂隙 3 处，合计 41.2 cm²」	H3D-Lab 分割输出 + M4 的 UV 与可见性
X4	病害语义时序追踪	把两期量化结果连成「这条裂缝一年扩展了 2.1 mm」	X3 + X1 + 跨期配准
X5	修复干预前后评估	量化「补配 18.4 cm²、色差降到 ΔE 3.1」	X1 + X2 + X3
X6	轻量化展示交付	GLB + LOD + Web 查看器，供展览与远程会诊	M6 已有 GLB 出口

7.1 X3 病害语义分割与图斑量化

这一步是全部延伸的地基：**先把病害从像素变成面积。**H3D-Lab 在照片空间给出类别掩膜（裂隙 / 剥落 / 盐析 / 生物 / 人为划痕，类别体系可映射）；H3D 这边负责把掩膜搬到几何上，用的是 M4 已经建好的那条通路：每张照片的位姿、内参、像素换算系数（pixel_scale）与 Z-buffer 可见性都在，把掩膜按可见性加权投影到 UV 图集，每个纹素做类别票决，得到一张**纹理空间的类别图。**

面积换算不需要新东西：UV 光栅化时每个纹素对应的三维面积 = 三角形面积除以该三角形覆盖的纹素数。把类别图上同类纹素的面积加起来就是真实面积——前提是尺度确定，否则只有相对面积。

输入	处理	输出
00_photo/<件>/*.jpg + H3D-Lab 掩膜（每张照片一张，类别 id）	可见性加权投影 → UV 图集类别票决 → 连通域分析 → 面积积分	<件>_defects.png（类别图）、<件>_defects.csv（类别 / 图斑数 / 面积 / 占比 / 最大图斑位置）

> **最容易写错的地方**：把照片掩膜**单视图硬投影**到几何上。背面、被遮挡面、掠射角面都会被打上病害，结果是「整件都有裂缝」。必须复用 M4 的可见性掩膜并做多视图票决——这和贴图是同一个问题，同一个解法。

7.2 X4 病害语义时序追踪（主线）

X3 给出「这一期有什么」，X4 给出「和上一期比，它怎么了」。本质上是三个已有能力的串联：X3 的图斑、X1 的几何差分，以及一个图斑级关联器。



图 7-1 X4 病害语义时序追踪的数据流

为什么 IoU + 质心 + 面积三个特征要一起用：只靠 IoU，「一条裂缝沿走向延伸」的 IoU 通常低于 0.5，会被误判成「旧裂缝消失 + 新裂缝出现」；加入质心距离（延伸时位移小）与面积比（单调增大）就能判成「扩展」。反过来剥落是「面积增大且质心基本不动」，盐析是「面积增大但边界模糊」，这些都可以写成规则，不必一上来就上学习模型。

状态	判据（建议初值）	报表列
新生	t2 有、t1 无（IoU 小于 0.1 且距离大于 2 倍等效半径）	t2 面积、位置
扩展	IoU 不小于 0.3 且面积比大于 1.15 且质心位移小于 1 倍等效半径	Δ面积、Δmm、速率
缩小	IoU 不小于 0.3 且面积比小于 0.85	Δ面积、位置
消失	t1 有、t2 无	原面积、位置
稳定	IoU 不小于 0.5 且面积比在 0.85 到 1.15 之间	仅登记

7.3 X5 修复干预前后评估

X5 是 X1 + X2 + X3 的组合应用，不引入新算法，只引入新的报表口径：清除量（干预前缺陷面积减干预后缺陷面积）、补配量（新增完好区域）、色差改善（前后 ΔE 差）、形貌变化（M3C2 的带符号距离，区分「削掉」与「补上」）。它的价值在于把「修复记录」从文字描述变成可核对的数字，这在馆藏档案里目前基本是空白。

7.4 X6 轻量化展示交付

M6 已经能出 GLB，但只有一个 LOD。展览与远程会诊要的是「手机能打开、能转、能看细节」：GLB 加 Draco 或 meshopt 压缩、三级 LOD（展示 / 常规 / 病害检查）、以及一个零安装的 Web 查看器。技术基础是 glTF 2.0 + PBR + 压缩扩展，与现有贴图管线不冲突，属于交付形态的扩展，风险最低、见效最快。

8 病害语义时序追踪：落地方案

这一章把 7.2 展开成可排期的任务。原则是**不新建管线**：能用 M4 的通路就用，能用 X1 的差分就用，新写的只有「分割结果接入」与「图斑关联」两块。

8.1 六步任务分解

步	任务	实现要点	产出
1	分割结果接入	约定目录 00_seg/<件>/<照片名>.png（类别 id 写在调色板或同名 json），不做模型推理，只做读取与校验	分割适配层 + 类别映射表
2	掩膜转 UV 类别图	复用 M4 的位姿 / 内参 / pixel_scale / Z-buffer；可见性加权票决	UV 类别图 + 每类纹素数
3	面积积分	UV 光栅化时记录每纹素三维面积；无尺度时同时给相对占比	defects.csv（面积或占比）
4	图斑连通域	UV 空间 8 邻域连通，形态学开运算去毛刺；输出面积、周长、质心、紧致度	图斑表
5	跨期关联	对齐 → IoU / 质心 / 面积比三特征 → 匈牙利匹配 → 状态判定	tracking.csv
6	报表与 GUI	时序曲线 + 图斑叠加视图；GUI 新增「病害演变」页签	报表章节 + 图形界面

8.2 三个关键设计决策

决策一：对齐用 UV 还是用几何？优先 UV。同一条管线的 UV 布局由几何指纹决定，缓存键已经能把「同几何」认出来；几何变了 UV 就会变，此时退回「几何 ICP 后按最近面片映射」。两条路都要留痕，报表里要能看出这一期用的是哪条。

决策二：显著性门槛谁来定？几何变化用 X1 的 M3C2 LoD95；语义变化用面积变化率加连通域最小面积（比如少于 4 个纹素的图斑直接丢弃，否则分割噪声会变成「每天都有新裂缝」）。

决策三：分割模型版本要不要进元数据？要。分割结果和模型版本强相关，不同版本对同一张照片可能给出不同类别。把模型名与版本写进 manifest，是让「两期对比」成立的前提。

8.3 分期与风险

期	范围	交付判据	主要风险
P1（约 2 周）	单类别（裂隙）端到端：分割接入 → UV 类别图 → 面积表	同一件跑两遍，面积差异小于 5%	可见性票决做错 → 背面被判成病害
P2（约 3 周）	图斑关联 + 状态机 + tracking.csv + 报表图	人工标注 20 个图斑，状态判定一致率不低于 85%	裂缝延伸被误判为「消失 + 新生」
P3（约 2 周）	GUI 病害演变页签 + 多类别扩展 + 论文实验表	3 件不同材质（陶 / 青铜 / 漆木）各跑通一次	多材质下分割类别漂移

与论文的关系：X4 的可辩护点不是「又训了一个分割模型」，而是**把几何、尺度、语义三条链条在同一个 UV 空间里对齐，并用显著性门槛把噪声挡在结论之外。**这条链路上每一步都有可量化的验收指标，比单纯报一个 mIoU 更经得起追问。

9 验证与发行

项	结果
全量测试	856 passed（1.1.10 为 843，净增 13：预览容错 12 + 交付格式 1）
真机复现	同一项目在源码版与打包版均立即出图；删掉 PLY 只留 OBJ 时副本回退 1.7 s 读完 11.7 万面
版本号	1.1.11（PE 资源、欢迎页、报表页脚三处一致）
未改动	几何与贴图算法、阶段顺序、配置项语义

发行链与上一版一致：PyInstaller 打包 exe → 建发行目录 → 更新 README / VERSION → 编译 Inno 安装包 → 回填 SHA-1 → 同步源码快照 → 打 zip → 打包版自证（--version 与 --batch 各跑一次）。

10 下一步

① **用 5.5 的第 ② 步把配准测出来**：对 PLY 施加已知刚体变换再让 M2 恢复。这是目前唯一还没有端到端证据的环节，也是论文里最容易被追问的地方。

② **补一件真实扫描件**：哪怕只借一次，也要让「双源融合」与「毫米级精度」有一次真实数据背书；SfM 自喂只能证明流程通，证明不了精度。

③ **X3 的最小闭环**：单类别裂隙、单件、跑两遍看面积稳定性。这一步做完，X4 的关联器才有输入可调。

④ **色卡常态化**：把「每件至少一张带色卡的照片」写进采集规范，否则 X2 长期停在 texture-random 模式，ΔE 永远是摆设。

⑤ **路径规范**：本次缺陷源于中文路径。建议在文档里明确「项目路径可用中文，但请避免同时使用全角标点与超长路径」，并在 GUI 新建项目时做一次校验提示。